

LAPORAN PRAKTIKUM
STUKTUR DATA

Algoritma Sorting (Shell Sort, Quick Sort, Merge Sort)
dan Bubble menggunakan GUI

Disusun oleh:
Thaariq Salam
2511532022

Dosen Pengampu: Dr. Wahyudi S.T.M.T
Asisten Dosen: Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan kehadirat Allah SWT, karena atas rahmat, hidayah, serta karunia-Nya, penulis dapat menyelesaikan Laporan Praktikum Struktur Data Pekan 8 dengan topik "Algoritma Sorting (Shell, Quick, dan Merge) pada Manajemen Playlist Lagu" ini dengan baik dan tepat pada waktunya.

Laporan ini disusun guna memenuhi salah satu tugas akademik pada mata kuliah Praktikum Struktur Data di Fakultas Teknologi Informasi, Universitas Andalas. Melalui laporan ini, diharapkan pembaca dapat memahami implementasi nyata dari konsep *Abstract Data Type* (ADT) serta perbandingan karakteristik, langkah kerja, hingga efisiensi dari berbagai algoritma pengurutan data tingkat lanjut berbasis larik objek dalam bahasa pemrograman Java.

Dalam proses penyusunan laporan praktikum ini, penulis banyak mendapatkan bimbingan, arahan, serta dukungan dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Bapak Dr. Wahyudi S.T.M.T selaku Dosen Pengampu mata kuliah Struktur Data.
2. Kak Sherly Sukmadira selaku Asisten Praktikum yang telah memberikan arahan selama pelaksanaan praktikum.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan, baik dari segi penyusunan kalimat, tata bahasa, maupun kedalaman analisis algoritma yang disajikan. Oleh karena itu, penulis sangat mengharapkan kritik, saran, dan masukan yang membangun dari para pembaca demi perbaikan dan penyempurnaan karya-karya tulis penulis di masa yang akan datang.

Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat nyata, menambah wawasan, serta menjadi referensi yang berguna bagi penulis khususnya, dan bagi para pembaca yang sedang mendalami studi struktur data dan algoritma pada umumnya.

Padang, 5 Juni 2026

Thaariq Salam

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	2
1.3 Manfaat.....	3
BAB II PEMBAHASAN.....	4
2.1 Class ShellSort_2511532022.java.....	4
2.1.1 Kode Program.....	4
2.1.2 Langkah Kerja.....	5
2.1.3 Analisis.....	6
2.2 Class QuickSort_2511532022.java.....	6
2.2.1 Kode Program.....	7
2.2.2 Langkah Kerja.....	8
2.2.3 Analisis.....	9
2.3 Class MergeSort_2511532022.java.....	10
2.3.1 Kode program.....	10
2.3.2 Langkah Kerja.....	11
2.3.3 Analisis.....	13
2.4 Class MergeSortGUI_2511532022.java.....	13
2.4.1 Kode program.....	14
2.4.2 Langkah kerja dan Penjelasan Kode.....	16
2.4.3 Analisis.....	18
2.5 Class BubbleSortGUI_2511532022.java.....	20
2.5.1 Kode program.....	20
2.5.2 Langkah kerja dan Penjelasan Kode.....	23
2.5.3 Analisis.....	25
BAB III PENUTUP.....	27
Tugas Praktikum Struktur Data Pekan 8.....	28
1. Kode Program.....	28
1.1 Class Lagu_2511532022.java (ADT).....	28
1.2 Class Sorting_2511532022.java.....	29
2. Langkah Kerja.....	31
3. Hasil Pengujian (<i>Output</i>).....	33

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia informatika dan ilmu komputer, pemrosesan data merupakan salah satu pilar utama dalam membangun sistem perangkat lunak yang efisien. Struktur data menyediakan metode yang terorganisasi untuk menyimpan, mengelola, dan memanipulasi informasi digital di dalam memori komputer. Salah satu operasi yang paling fundamental dan paling sering dilakukan terhadap kumpulan data adalah pengurutan (*sorting*). Pengurutan didefinisikan sebagai proses menyusun kembali sekumpulan objek atau data ke dalam aturan logis tertentu, baik secara berurutan naik (*ascending*) dari nilai terkecil ke nilai terbesar, maupun berurutan turun (*descending*) dari nilai terbesar ke nilai terkecil. Keberadaan algoritma pengurutan yang optimal sangat krusial karena data yang terstruktur secara berurutan akan secara signifikan mempercepat proses pencarian (*searching*) serta mempermudah analisis data tingkat lanjut.

Seiring dengan perkembangan kompleksitas komputasi, para ilmuwan komputer telah merancang berbagai variasi algoritma pengurutan yang masing-masing memiliki karakteristik, mekanisme pertukaran elemen, serta efisiensi memori yang berbeda. Algoritma konvensional seperti *Bubble Sort* dan *Shell Sort* menawarkan pendekatan yang relatif sederhana dan intuitif, yang sangat cocok digunakan untuk memahami logika dasar dari penataan indeks elemen dalam memori linear. Di sisi lain, ketika menghadapi volume data yang berskala besar, efisiensi waktu eksekusi menjadi prioritas utama. Oleh karena itu, dikembangkan pula pendekatan berbasis paradigma algoritma yang lebih kompleks seperti *Divide and Conquer* (bagi dan taklukkan) yang diimplementasikan secara elegan dalam metode *Quick Sort* dan *Merge Sort*. Memahami kompleksitas waktu (*time complexity*) dan

kompleksitas ruang (*space complexity*) dari variasi algoritma ini adalah kompetensi wajib bagi seorang mahasiswa informatika.

Selain efisiensi berbasis teks di dalam konsol (*command-line interface*), aspek interaksi pengguna atau *User Interface* (UI) juga memegang peranan yang sangat penting dalam pengembangan perangkat lunak modern. Visualisasi sebuah algoritma yang abstrak menjadi bentuk grafis yang dinamis dapat membantu pengguna maupun pengembang dalam memahami perilaku internal sistem secara *real-time*. Melalui pemanfaatan pustaka Java Swing, visualisasi pergerakan data dari algoritma pengurutan dapat diimplementasikan ke dalam bentuk *Graphical User Interface* (GUI). Dengan antarmuka berbasis GUI ini, proses pertukaran elemen, pembagian sub-array, dan kondisi akhir data dapat diamati secara bertahap langkah demi langkah. Berdasarkan urgensi pemahaman teoretis algoritma pengurutan dan implementasi praktis antarmuka visualnya, maka laporan praktikum struktur data ini disusun sebagai dokumentasi komprehensif terhadap pengerjaan modul pengurutan.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut

1. Memahami konsep dasar, mekanisme kerja, serta karakteristik dari berbagai macam algoritma pengurutan data meliputi *Shell Sort*, *Quick Sort*, dan *Merge Sort*.
2. Mengimplementasikan algoritma pengurutan data secara terstruktur menggunakan bahasa pemrograman Java berdasarkan kaidah penulisan kode yang bersih dan efisien.
3. Membangun aplikasi visual berbasis *Graphical User Interface* (GUI) dengan pustaka Java Swing untuk mensimulasikan eksekusi langkah demi langkah pada algoritma *Merge Sort* dan *Bubble Sort*.

4. Menganalisis perbandingan performa, kelebihan, kekurangan, serta efisiensi dari masing-masing algoritma pengurutan yang diuji melalui pengamatan logika alur program.

1.3 Manfaat

Manfaat yang didapat dari praktikum ini adalah sebagai berikut.

1. Mahasiswa dapat mengasah kemampuan berpikir logis dan analitis dalam memecahkan masalah manipulasi urutan data linear secara sistematis.
2. Mahasiswa memperoleh pengalaman praktis dalam merancang desain antarmuka pengguna berbasis desktop yang interaktif dan responsif menggunakan elemen-elemen Java Swing.
3. Menyediakan referensi implementasi kode program pengurutan yang valid dan terstruktur bagi pengembangan sistem atau riset akademis di bidang pemrograman berorientasi objek.
4. Membantu mempermudah visualisasi pembelajaran konsep abstrak algoritma pengurutan yang rumit menjadi proses perbandingan elemen yang mudah dipahami secara visual.

BAB II

PEMBAHASAN

2.1 Class ShellSort_2511532022.java

Class ini merupakan implementasi dari algoritma *Shell Sort* yang melakukan pengurutan data dengan cara membandingkan elemen-elemen yang terpisah oleh jarak (*gap*) tertentu yang kemudian diperkecil secara bertahap hingga bernilai satu.

2.1.1 Kode Program

```
1 package pekan8_2511532022;
2
3 public class ShellSort_2511532022 {
4     public static void shellSort(int[] A_2022) {
5         int n_2022 = A_2022.length;
6         int gap_2022 = n_2022 / 2;
7
8         while (gap_2022 > 0) {
9             for (int i_2022 = gap_2022; i_2022 < n_2022; i_2022++) {
10                 int temp_2022 = A_2022[i_2022];
11                 int j_2022 = i_2022;
12                 while (j_2022 >= gap_2022 && A_2022[j_2022 - gap_2022] > temp_2022) {
13                     A_2022[j_2022] = A_2022[j_2022 - gap_2022];
14                     j_2022 = j_2022 - gap_2022;
15                 }
16                 A_2022[j_2022] = temp_2022;
17             }
18             gap_2022 = gap_2022 / 2;
19         }
20     }
21     public static void main(String[] args_2022) {
22         int[] data_2022 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
23
24         System.out.print("Sebelum: ");
25         printArray(data_2022);
26
27         shellSort(data_2022);
28
29         System.out.print("Sesudah (shell sort): ");
30         printArray(data_2022);
31     }
32
33     public static void printArray(int[] arr_2022) {
34         for (int i_2022 : arr_2022) {
35             System.out.print(i_2022 + " ");
36         }
37         System.out.println();
38     }
39 }
```


2.1.2 Langkah Kerja

Berikut adalah penjelasan detail fungsi dari setiap bagian kode di atas.

- Program memulai eksekusi dari dalam method main, di mana sebuah array integer bernama `data_2022` diinisialisasi dengan beberapa elemen acak.
- Method `printArray` dipanggil pertama kali untuk menampilkan urutan susunan elemen mentah sebelum dilakukan proses pengurutan ke konsol sistem.
- Program memanggil method statis `shellSort` dengan melempar objek array `data_2022` sebagai argumen utama pengurutan.
- Di dalam method `shellSort`, panjang array dihitung dan disimpan ke dalam variabel `n_2022`, lalu jarak awal ditentukan dengan rumus $gap_2022 = n_2022 / 2$.
- Sebuah struktur perulangan `while` berjalan selama nilai `gap_2022` masih lebih besar dari nol untuk mengendalikan fase penyusutan jarak perbandingan.
- Di dalam perulangan `gap` tersebut, struktur perulangan `for` melakukan pemindaian elemen dimulai dari indeks sebesar nilai `gap_2022` menuju ke ujung kanan array.
- Nilai elemen saat ini disimpan ke dalam variabel temporer `temp_2022`, dan indeksnya disalin ke dalam variabel pelacak posisi elemen kiri yaitu `j_2022`.
- Perulangan `while` bagian dalam membandingkan elemen berjarak `gap`; jika elemen di posisi kiri (`j_2022 - gap_2022`) lebih besar dari `temp_2022`, elemen tersebut digeser ke kanan mengisi posisi `j_2022`.
- Proses pergeseran terus berulang dan nilai indeks `j_2022` dikurangi sejauh ukuran `gap` hingga kondisi perbandingan tidak lagi terpenuhi.
- Nilai di dalam variabel `temp_2022` ditempatkan ke posisi akhir indeks `j_2022` yang telah dikondisikan.

- Setelah satu siklus perulangan for selesai untuk seluruh bagian elemen array, nilai variabel *gap_2022* diperkecil dengan cara dibagi dua (*gap_2022* / 2).
- Seluruh siklus di atas terus berulang hingga *gap* bernilai 0, lalu kendali kembali ke method main untuk mencetak hasil akhir array yang telah terurut menggunakan method *printArray*.

2.1.3 Analisis

Algoritma *Shell Sort* merupakan sebuah optimasi yang sangat signifikan dari algoritma *Insertion Sort* konvensional. Keunggulan utamanya terletak pada kemampuannya untuk melakukan pertukaran elemen yang terpisah jarak jauh pada fase awal perulangan, sehingga mempercepat elemen kecil untuk bergeser ke arah kiri array tanpa harus melalui pertukaran bertahap satu demi satu indeks bertetangga.

Penentuan sekuens ukuran *gap* memegang kendali penuh atas kompleksitas waktu dari performa eksekusi kode ini. Berdasarkan kode di atas yang menggunakan pembagian biner standar ($n/2$), performa rata-rata yang dihasilkan berkisar pada $O(n^{1.25})$ hingga $O(n^2)$ pada skenario terburuk, menjadikannya jauh lebih efisien daripada *Bubble Sort* biasa tanpa memerlukan alokasi memori tambahan yang besar karena pengurutan dilakukan langsung di tempat (*in-place sorting*).

```
<terminated> ShellSort_2511532022 [Java Application]
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (shell sort): 1 2 3 4 5 6 7 8 9 10
```

2.2 Class QuickSort_2511532022.java

Class ini mengimplementasikan algoritma *Quick Sort* berbasis paradigma *divide and conquer* dengan optimasi pemilihan elemen pivot menggunakan metode *Median-of-Three* untuk menghindari penurunan performa pada kondisi array yang hampir terurut.

2.2.1 Kode Program

```
1 package pekan8_2511532022;
2
3 public class QuickSort_2511532022 {
4
5     static void swap(int[] arr_2022, int i_2022, int j_2022) {
6         int temp_2022 = arr_2022[i_2022];
7         arr_2022[i_2022] = arr_2022[j_2022];
8         arr_2022[j_2022] = temp_2022;
9     }
10
11     // Metode tambahan untuk mengatur pivot menggunakan Median of three
12     static void medianOfThree(int[] arr_2022, int low_2022, int high_2022) {
13         int mid_2022 = low_2022 + (high_2022 - low_2022) / 2;
14
15         // Urutkan elemen low, mid, dan high
16         if (arr_2022[low_2022] > arr_2022[mid_2022]) {
17             swap(arr_2022, low_2022, mid_2022);
18         }
19         if (arr_2022[low_2022] > arr_2022[high_2022]) {
20             swap(arr_2022, low_2022, high_2022);
21         }
22         if (arr_2022[mid_2022] > arr_2022[high_2022]) {
23             swap(arr_2022, mid_2022, high_2022);
24         }
25         swap(arr_2022, mid_2022, high_2022);
26     }
27
28     static int partition(int[] arr_2022, int low_2022, int high_2022) {
29         // Panggil fungsi medianOfThree sebelum menentukan pivot
30         medianOfThree(arr_2022, low_2022, high_2022);
31
32         int pivot_2022 = arr_2022[high_2022];
33         int i_2022 = (low_2022 - 1);
34
35         // Menggunakan j < high agar logika partisi berjalan tepat
36         for (int j_2022 = low_2022; j_2022 < high_2022; j_2022++) {
37             // Jika elemen saat ini lebih kecil dari atau sama dengan pivot
38             if (arr_2022[j_2022] < pivot_2022) {
39                 // Increment indeks elemen yang lebih kecil
40                 i_2022++;
41                 swap(arr_2022, i_2022, j_2022);
42             }
43         }
44         swap(arr_2022, i_2022 + 1, high_2022);
45         return (i_2022 + 1);
46     }
47
48     static void quickSort(int[] arr_2022, int low_2022, int high_2022) {
49         if (low_2022 < high_2022) {
50             int pi_2022 = partition(arr_2022, low_2022, high_2022);
51             quickSort(arr_2022, low_2022, pi_2022 - 1);
52             quickSort(arr_2022, pi_2022 + 1, high_2022);
53         }
54     }
55
56     public static void printArr(int[] arr_2022) {
57         for (int i_2022 = 0; i_2022 < arr_2022.length; i_2022++) {
58             System.out.print(arr_2022[i_2022] + " ");
59         }
60         System.out.println();
61     }
62
63     public static void main(String[] args_2022) {
64         int[] arr_2022 = { 10, 7, 8, 9, 1, 5 };
65         int N_2022 = arr_2022.length;
66         System.out.print("Data sebelum diurutkan: ");
67         printArr(arr_2022);
68
69         quickSort(arr_2022, 0, N_2022 - 1);
70
71         System.out.print("Data Terurut quicksort: ");
72         printArr(arr_2022);
73     }
74 }
```

2.2.2 Langkah Kerja

Berikut adalah penjelasan fungsi baris-baris kode pada class di atas

- Program mengeksekusi blok method main, mendeklarasikan array `arr_2022` berisi elemen numerik acak, mencetak data awal, lalu memanggil method rujukan `quickSort`.
- Method `quickSort` menerima parameter berupa array utama, batas indeks terendah (`low_2022`), dan batas indeks tertinggi (`high_2022`).
- Kondisi pencabangan mendasar memeriksa apakah batas bawah masih lebih kecil dari batas atas (`low_2022 < high_2022`); jika benar, proses partisi dimulai dengan memanggil method `partition`.
- Di dalam method `partition`, fungsi pengkondisian optimasi bernama `medianOfThree` dipanggil terlebih dahulu untuk memproses posisi pivot yang ideal.
- Method `medianOfThree` mencari nilai tengah indeks, membandingkan serta menukar elemen pada posisi `low`, `mid`, dan `high` menggunakan fungsi pembantu `swap` agar elemen median berakhir di posisi indeks `high` sebagai elemen pivot utama.
- Kembali ke method `partition`, variabel `pivot_2022` diisi dengan nilai dari indeks terujung kanan (`arr_2022[high_2022]`) dan variabel penanda penempatan elemen kecil `i_2022` diinisialisasi senilai `low_2022 - 1`.
- Struktur perulangan `for` dengan variabel kontrol `j_2022` memindai area elemen dari batas bawah hingga sebelum elemen pivot (`high_2022 - 1`).
- Di setiap iterasi, jika elemen `arr_2022[j_2022]` bernilai lebih kecil dari elemen `pivot_2022`, nilai variabel penanda `i_2022` dinaikkan satu poin, lalu elemen pada indeks `i_2022` ditukar posisi dengan elemen pada indeks `j_2022` menggunakan method `swap`.

- Setelah perulangan for selesai membagi data, fungsi swap dijalankan sekali lagi untuk menempatkan elemen pivot dari indeks `high_2022` ke posisi aslinya di indeks `i_2022 + 1`.
- Method partition mengembalikan nilai posisi indeks pivot yang telah tetap (`i_2022 + 1`) ke dalam variabel tampung `pi_2022` di dalam fungsi pemanggil.
- Secara rekursif, method quickSort memanggil dirinya sendiri kembali untuk memproses sub-array bagian kiri (dari `low_2022` hingga `pi_2022 - 1`).
- Secara rekursif pula, method quickSort memanggil dirinya sendiri untuk memproses sub-array bagian kanan (dari `pi_2022 + 1` hingga `high_2022`) hingga seluruh data terurut sempurna dan dicetak oleh method `printArr`.

2.2.3 Analisis

Quick Sort terkenal sebagai salah satu algoritma pengurutan berbasis perbandingan yang paling cepat dalam praktiknya. Penggunaan taktik *Median-of-Three* pada kode ini merupakan langkah pencegahan yang sangat baik dari kelemahan bawaan *Quick Sort* tradisional yang rentan mengalami penurunan performa menjadi $O(n^2)$ apabila pivot yang dipilih secara naif selalu berupa nilai ekstrem (terkecil atau terbesar).

Melalui pencarian nilai median dari elemen pertama, tengah, dan terakhir, pembagian partisi dapat dijamin menjadi lebih seimbang mendekati proporsi merata. Hal ini memastikan struktur pemanggilan fungsi rekursif tetap terjaga pada kompleksitas waktu rata-rata $O(n \log n)$, yang beroperasi dengan sangat cepat karena minimnya konsumsi memori *overhead* eksternal.

```
<terminated> QuickSort_2511532022 [Java].
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10
```

2.3 Class MergeSort_2511532022.java

Class ini mengimplementasikan algoritma *Merge Sort* standar dengan membagi array secara rekursif menjadi sub-array yang lebih kecil, mengurutkannya, dan menggabungkannya kembali menggunakan array bantuan (*temporary array*) secara efisien.

2.3.1 Kode program

```
1 package pekan8_2511532022;
2
3 public class MergeSort_2511532022 {
4     void merge(int arr_2022[], int l_2022, int m_2022, int r_2022) {
5         // Find sizes of two subarrays to be merged
6         int n1_2022 = m_2022 - l_2022 + 1;
7         int n2_2022 = r_2022 - m_2022;
8
9         /* Create temp arrays */
10        int L_2022[] = new int[n1_2022];
11        int R_2022[] = new int[n2_2022];
12
13        /* Copy data to temp arrays */
14        for (int i_2022 = 0; i_2022 < n1_2022; ++i_2022) {
15            L_2022[i_2022] = arr_2022[l_2022 + i_2022];
16        }
17        for (int j_2022 = 0; j_2022 < n2_2022; ++j_2022) {
18            R_2022[j_2022] = arr_2022[m_2022 + 1 + j_2022];
19        }
20
21        int i_2022 = 0, j_2022 = 0;
22
23        // Initial index of merged subarray array
24        int k_2022 = l_2022;
25        while (i_2022 < n1_2022 && j_2022 < n2_2022) {
26            if (L_2022[i_2022] <= R_2022[j_2022]) {
27                arr_2022[k_2022] = L_2022[i_2022];
28                i_2022++;
29            } else {
30                arr_2022[k_2022] = R_2022[j_2022];
31                j_2022++;
32            }
33            k_2022++;
34        }
35
36        /* Copy remaining elements of L[] if any */
37        while (i_2022 < n1_2022) {
38            arr_2022[k_2022] = L_2022[i_2022];
39            i_2022++;
40            k_2022++;
41        }
42    }
43 }
```

```

42
43      /* Copy remaining elements of R[] if any */
44      while (j_2022 < n2_2022) {
45          arr_2022[k_2022] = R_2022[j_2022];
46          j_2022++;
47          k_2022++;
48      }
49  }
50
51  void sort(int arr_2022[], int l_2022, int r_2022) {
52      if (l_2022 < r_2022) {
53          // Find the middle point
54          int m_2022 = (l_2022 + r_2022) / 2;
55
56          // Sort first and second halves
57          sort(arr_2022, l_2022, m_2022);
58          sort(arr_2022, m_2022 + 1, r_2022);
59
60          // Merge the sorted halves
61          merge(arr_2022, l_2022, m_2022, r_2022);
62      }
63  }
64  /* A utility function to print array of size n */
65  static void printArray(int arr_2022[]) {
66      int n_2022 = arr_2022.length;
67      for (int i_2022 = 0; i_2022 < n_2022; ++i_2022)
68          System.out.print(arr_2022[i_2022] + " ");
69      System.out.println();
70  }
71  public static void main(String args_2022[]) {
72      int arr_2022[] = { 12, 11, 13, 5, 6, 7 };
73      System.out.println("Sebelum terurut");
74      printArray(arr_2022);
75
76      MergeSort_2511532022 ob_2022 = new MergeSort_2511532022();
77      ob_2022.sort(arr_2022, 0, arr_2022.length - 1);
78
79      System.out.println("\nSesudah Terurut menggunakan merge Sort");
80      printArray(arr_2022);
81  }
82  }

```

2.3.2 Langkah Kerja

Berikut adalah penjelasan fungsi setiap baris kodenya.

- Eksekusi diawali dari method main dengan pembuatan objek array bernama arr_2022 yang menampung buah elemen data integer tak terurut.
- Program mencetak isi data awal ke konsol, menginstansiasi objek ob_2022 dari kelas utama, lalu memicu eksekusi pemrosesan data lewat method sort.
- Method sort memeriksa kondisi awal pembagian; selama indeks kiri (l_2022) masih bernilai lebih kecil dari indeks kanan (r_2022), maka proses pembagian dilanjutkan.

- Nilai titik tengah array dihitung menggunakan rumus pembagian standar matematika yaitu $m_2022 = (l_2022 + r_2022) / 2$.
- Method sort memanggil dirinya sendiri secara rekursif untuk membelah dan mengkondisikan sub-array segmen kiri dari rentang indeks l_2022 hingga titik tengah m_2022 .
- Method sort memanggil dirinya kembali secara rekursif untuk mengisolasi sub-array segmen kanan dari posisi indeks $m_2022 + 1$ hingga batas terujung r_2022 .
- Setelah proses pembelahan mencapai tingkat elemen tunggal terkecil, program memanggil fungsi penggabungan elemen terurut yaitu method merge.
- Di dalam method merge, ukuran dari dua sub-array yang akan digabungkan dihitung dan disimpan ke dalam variabel ukuran temporary $n1_2022$ dan $n2_2022$.
- Dua buah struktur array pembantu eksternal dideklarasikan dan dialokasikan ke memori komputer dengan penamaan objek $L_2022[]$ dan $R_2022[]$.
- Dua buah perulangan loop for menyalin data numerik dari array induk utama menuju masing-masing array pembantu segmen kiri dan segmen kanan.
- Struktur perulangan kondisional while membandingkan nilai elemen pada indeks berjalan $L_2022[i_2022]$ dengan elemen $R_2022[j_2022]$. Elemen berukuran lebih kecil akan disalin kembali secara berurutan ke posisi indeks k_2022 pada array induk asli.
- Dua buah blok loop while penutup bertugas menyalin sisa elemen dari array pembantu L_2022 atau R_2022 yang belum sempat dimasukkan akibat selisih ukuran, hingga data terurut utuh dan dicetak kembali di baris akhir method main.

2.3.3 Analisis

Merge Sort menawarkan sebuah karakteristik keunggulan performa pengurutan yang sangat konsisten di segala jenis skenario data input. Berbeda dengan *Quick Sort* yang performanya dipengaruhi oleh pemilihan komponen pivot, algoritma *Merge Sort* menjamin kompleksitas waktu tetap berada pada nilai mutlak $O(n \log n)$, baik pada kondisi data acak, data terurut terbalik, maupun data yang sudah rapi sejak awal. Kelemahan mutlak yang melekat pada kode ini adalah aspek efisiensi alokasi memori (*space complexity*). Karena proses penggabungan membutuhkan pembuatan struktur array pembantu baru (*L_2022* dan *R_2022*) di setiap pemanggilan method merger, algoritma ini mengonsumsi ruang memori tambahan sebesar $O(n)$, menjadikannya kurang ideal untuk sistem embedded dengan keterbatasan ruang RAM

```
<terminated> MergeSort_2511532022 [Java ,  
Sebelum terurut  
12 11 13 5 6 7  
  
Sesudah Terurut menggunakan merge Sort  
5 6 7 11 12 13
```

2.4 Class MergeSortGUI_2511532022.java

Class ini merupakan bentuk pengembangan visual interaktif dari algoritma *Merge Sort* yang memanfaatkan komponen Java Swing untuk memetakan proses penguraian dan penggabungan elemen array dalam bentuk simulasi grafis bertahap.

2.4.1 Kode program

```
1 package pekan8_2511532022;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.FlowLayout;
7 import java.awt.Font;
8
9 import javax.swing.BorderFactory;
10 import javax.swing.JButton;
11 import javax.swing.JFrame;
12 import javax.swing.JLabel;
13 import javax.swing.JOptionPane;
14 import javax.swing.JPanel;
15 import javax.swing.JScrollPane;
16 import javax.swing.JTextArea;
17 import javax.swing.JTextField;
18 import javax.swing.SwingConstants;
19 import javax.swing.SwingUtilities;
20
21 import java.util.LinkedList;
22 import java.util.Queue;
23
24 public class MergeSortGUI_2511532022 extends JFrame {
25
26     private static final long serialVersionUID = 1L;
27     private int[] array_2022;
28     private JLabel[] labelArray_2022;
29     private JButton stepButton_2022, resetButton_2022, setButton_2022;
30     private JTextField inputField_2022;
31     private JPanel panelArray_2022;
32     private JTextArea stepArea_2022;
33
34     private Queue<int[]> mergeQueue_2022 = new LinkedList<>();
35     private int stepCount_2022 = 1;
36     private boolean isMerging_2022 = false;
37     private boolean copying_2022 = false;
38
39     private int[] temp_2022;
40     private int left_2022, mid_2022, right_2022;
41     private int i_2022, j_2022, k_2022;
42
43     public MergeSortGUI_2511532022() {
44         setTitle("Merge Sort Langkah per Langkah");
45         setSize(750, 400);
46         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
47         setLocationRelativeTo(null);
48         setLayout(new BorderLayout());
49
50         // panel input
51         JPanel inputPanel_2022 = new JPanel(new FlowLayout());
52         inputField_2022 = new JTextField(30);
53         setButton_2022 = new JButton("Set Array");
54         inputPanel_2022.add(new JLabel("Masukan angka (pisahkan dengan koma)"));
55         inputPanel_2022.add(inputField_2022);
56         inputPanel_2022.add(setButton_2022);
57
58         // panel array visual
59         panelArray_2022 = new JPanel();
60         panelArray_2022.setLayout(new FlowLayout());
61
62         // panel kontrol
63         JPanel controlPanel_2022 = new JPanel();
64         stepButton_2022 = new JButton("Langkah selanjutnya");
65         resetButton_2022 = new JButton("Reset");
66         controlPanel_2022.add(stepButton_2022);
67         controlPanel_2022.add(resetButton_2022);
68
69         // area text untuk log langkah langkah
70         stepArea_2022 = new JTextArea(8, 60);
71         stepArea_2022.setEditable(false);
72         stepArea_2022.setFont(new Font("Monospaced", Font.PLAIN, 14));
73         JScrollPane scrollPane_2022 = new JScrollPane(stepArea_2022);
74
75         // tambahkan panel ke frame
76         add(inputPanel_2022, BorderLayout.NORTH);
77         add(panelArray_2022, BorderLayout.CENTER);
78         add(controlPanel_2022, BorderLayout.SOUTH);
79         add(scrollPane_2022, BorderLayout.EAST);
80
81         // event set array
82         setButton_2022.addActionListener(e -> setArrayFromInput_2022());
83     }
```

```

84 // event selanjutnya
85 stepButton_2022.addActionListener(e -> performStep_2022());
86
87 // event reset
88 resetButton_2022.addActionListener(e -> reset_2022());
89 }
90
91 private void setArrayFromInput_2022() {
92     String text_2022 = inputField_2022.getText().trim();
93     if (text_2022.isEmpty()) return;
94     String[] parts_2022 = text_2022.split(",");
95     array_2022 = new int[parts_2022.length];
96     try {
97         for (int k = 0; k < parts_2022.length; k++) {
98             array_2022[k] = Integer.parseInt(parts_2022[k].trim());
99         }
100     } catch (NumberFormatException e) {
101         JOptionPane.showMessageDialog(this, "Masukkan hanya angka dengan koma!",
102             "Error", JOptionPane.ERROR_MESSAGE);
103         return;
104     }
105
106     labelArray_2022 = new JLabel[array_2022.length];
107     panelArray_2022.removeAll();
108     for (int k = 0; k < array_2022.length; k++) {
109         labelArray_2022[k] = new JLabel(String.valueOf(array_2022[k]));
110         labelArray_2022[k].setFont(new Font("Arial", Font.BOLD, 24));
111         labelArray_2022[k].setOpaque(true);
112         labelArray_2022[k].setBackground(Color.WHITE);
113         labelArray_2022[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
114         labelArray_2022[k].setPreferredSize(new Dimension(50, 50));
115         labelArray_2022[k].setHorizontalAlignment(SwingConstants.CENTER);
116         panelArray_2022.add(labelArray_2022[k]);
117     }
118
119     mergeQueue_2022.clear();
120     generateMergeSteps_2022(0, array_2022.length - 1);
121     stepButton_2022.setEnabled(true);
122     stepArea_2022.setText("");
123     stepCount_2022 = 1;
124     isMerging_2022 = false;
125

```

```

126     panelArray_2022.revalidate();
127     panelArray_2022.repaint();
128 }
129
130 private void generateMergeSteps_2022(int left, int right) {
131     if (left >= right) return;
132     int mid = (left + right) / 2;
133     generateMergeSteps_2022(left, mid);
134     generateMergeSteps_2022(mid + 1, right);
135     mergeQueue_2022.add(new int[]{left, mid, right});
136 }
137
138 private void performStep_2022() {
139     resetHighlights_2022();
140
141     if (!isMerging_2022 && !mergeQueue_2022.isEmpty()) {
142         int[] range_2022 = mergeQueue_2022.poll();
143         left_2022 = range_2022[0];
144         mid_2022 = range_2022[1];
145         right_2022 = range_2022[2];
146         temp_2022 = new int[right_2022 - left_2022 + 1];
147         i_2022 = left_2022;
148         j_2022 = mid_2022 + 1;
149         k_2022 = 0;
150         copying_2022 = false;
151         isMerging_2022 = true;
152         stepArea_2022.append("Langkah " + stepCount_2022++ +
153             ": Mulai merge dari " + left_2022 + " ke " + right_2022 + "\n");
154         return;
155     }
156
157     if (isMerging_2022 && !copying_2022) {
158         if (i_2022 <= mid_2022 && j_2022 <= right_2022) {
159             labelArray_2022[i_2022].setBackground(Color.CYAN);
160             labelArray_2022[j_2022].setBackground(Color.CYAN);
161             if (array_2022[i_2022] <= array_2022[j_2022]) {
162                 temp_2022[k_2022++] = array_2022[i_2022++];
163             } else {
164                 temp_2022[k_2022++] = array_2022[j_2022++];
165             }
166             stepArea_2022.append("Langkah " + stepCount_2022++ + ": Bandingkan dan salin elemen\n");
167             return;

```

```

168~    } else if (i_2022 <= mid_2022) {
169        temp_2022[k_2022++] = array_2022[i_2022++];
170        stepArea_2022.append("Langkah " + stepCount_2022++ + ": Salin sisa kiri\n");
171        return;
172~    } else if (j_2022 <= right_2022) {
173        temp_2022[k_2022++] = array_2022[j_2022++];
174        stepArea_2022.append("Langkah " + stepCount_2022++ + ": Salin sisa kanan\n");
175        return;
176~    } else {
177        copying_2022 = true;
178        k_2022 = 0;
179        return;
180    }
181}
182
183~    if (copying_2022 && k_2022 < temp_2022.length) {
184        array_2022[left_2022 + k_2022] = temp_2022[k_2022];
185        labelArray_2022[left_2022 + k_2022].setText(String.valueOf(temp_2022[k_2022]));
186        labelArray_2022[left_2022 + k_2022].setBackground(Color.GREEN);
187        k_2022++;
188        stepArea_2022.append("Langkah " + stepCount_2022++ + ": Tempelkan ke array utama\n");
189        return;
190    }
191
192~    if (copying_2022 && k_2022 == temp_2022.length) {
193        isMerging_2022 = false;
194        copying_2022 = false;
195    }
196
197~    if (mergeQueue_2022.isEmpty() && !isMerging_2022) {
198        stepArea_2022.append("Selesai.\n");
199        stepButton_2022.setEnabled(false);
200        JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
201    }
202}
203
204~    private void resetHighlights_2022() {
205        if (labelArray_2022 == null) return;
206~        for (JLabel label : labelArray_2022) {
207            label.setBackground(Color.WHITE);
208        }
209    }

```

```

210
211~    private void reset_2022() {
212        inputField_2022.setText("");
213        panelArray_2022.removeAll();
214        panelArray_2022.revalidate();
215        panelArray_2022.repaint();
216        stepArea_2022.setText("");
217        stepButton_2022.setEnabled(false);
218        mergeQueue_2022.clear();
219        isMerging_2022 = false;
220        stepCount_2022 = 1;
221    }
222
223~    public static void main(String[] args) {
224~        SwingUtilities.invokeLater(() -> {
225            MergeSortGUI_2511532022 gui = new MergeSortGUI_2511532022();
226            gui.setVisible(true);
227        });
228    }
229}

```

2.4.2 Langkah kerja dan Penjelasan Kode

Berikut adalah penjelasan fungsi setiap baris kodenya

- Program diinisialisasi melalui method main yang memicu pembuatan komponen GUI di dalam thread khusus Java melalui perintah terstruktur `SwingUtilities.invokeLater`.

- Konstruktor utama mempersiapkan bingkai jendela aplikasi desktop (JFrame), menetapkan judul window, skema tata letak komponen BorderLayout, serta memasang panel input di sisi atas, panel visualisasi di bagian tengah, tombol navigasi di bagian bawah, dan log teks berbasis JTextArea di sisi kanan.
- Ketika pengguna mengetik urutan data angka acak dipisahkan koma dan menekan tombol "Set Array", method `setArrayFromInput_2022` dipanggil untuk memvalidasi teks input menggunakan mekanisme penanganan eksepsi try-catch terhadap galat parsing data bukan angka.
- Setelah input dinyatakan valid, array grafis berupa deretan objek JLabel dengan latar belakang putih dan garis tepi hitam dibuat secara dinamis di dalam kontainer `panelArray_2022` untuk merepresentasikan masing-masing angka secara visual.
- Method `generateMergeSteps_2022` dieksekusi secara internal untuk mensimulasikan pembagian indeks rentang data secara logis menggunakan prinsip rekursif, kemudian menyimpan setiap koordinat batas penggabungan (indeks kiri, tengah, dan kanan) ke dalam struktur data antrean linier berwujud objek `mergeQueue_2022`.
- Ketika pengguna menekan tombol "Langkah selanjutnya", program mengeksekusi method `performStep_2022` yang pertama-tama membersihkan seluruh warna penanda pada komponen visual melalui pemanggilan method pembantu `resetHighlights_2022`.
- Jika status penggabungan (`isMerging_2022`) bernilai salah dan antrean langkah di dalam `mergeQueue_2022` tidak kosong, program mengambil koordinat batas pengerjaan dari antrean menggunakan fungsi `poll()`, mengalokasikan memori array pembantu temporer `temp_2022`, mengeset pointer indeks pembandingan, dan menuliskan log pembukaan proses penggabungan baru ke area teks.

- Pada ketukan tombol langkah berikutnya, saat proses penggabungan aktif, program membandingkan nilai elemen pada array utama di posisi indeks `i_2022` dan `j_2022`, memberikan warna biru muda (Cyan) pada kedua komponen JLabel yang sedang dibandingkan, lalu menyalin nilai yang lebih kecil ke dalam array temporer. Log perbandingan kemudian dicetak ke layar log teks.
- Jika salah satu sisi sub-array telah habis dibandingkan, penekanan tombol langkah selanjutnya akan secara otomatis menyalin sisa elemen dari sisi array yang masih tersisa menuju struktur array temporer hingga penuh.
- Setelah array temporer terisi lengkap, status program berubah menjadi fase penyalinan balik (`copying_2022 = true`). Setiap kali tombol ditekan, satu buah nilai dari array temporer dipindahkan kembali ke array utama secara nyata. Komponen JLabel yang diperbarui akan berubah warna latarnya menjadi hijau cerah untuk menandakan penulisan data berhasil, dan teks angka label diperbarui.
- Setelah rentang elemen tersebut selesai disalin kembali seluruhnya, status penggabungan dinonaktifkan sementara dan sistem bersiap mengambil antrian pengerjaan berikutnya dari objek `mergeQueue_2022`.
- Apabila antrian langkah telah habis total dan seluruh proses penggabungan internal telah diselesaikan, tombol navigasi dinonaktifkan otomatis dan sebuah kotak dialog pop-up pesan informasi berupa perintah `JOptionPane.showMessageDialog` muncul di layar mengonfirmasi bahwa seluruh rangkaian proses *Merge Sort* telah selesai. Tombol "Reset" dapat ditekan kapan saja untuk mengosongkan seluruh panel dan mengembalikan variabel ke kondisi nol melalui method `reset_2022`.

2.4.3 Analisis

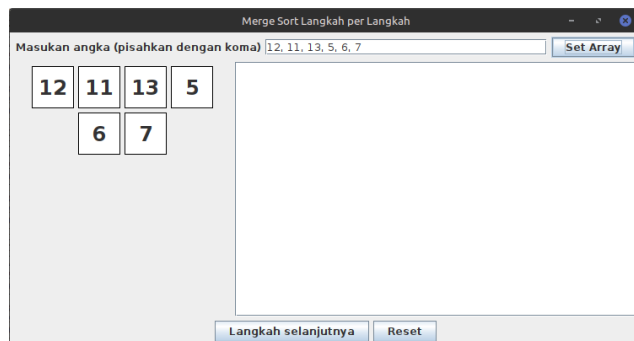
Implementasi aplikasi simulasi *Merge Sort* GUI ini memiliki keunikan struktural yang sangat menarik pada bagian manajemen alur eksekusinya. Pada kelas

berbasis teks konvensional, proses *Merge Sort* berjalan secara otomatis, cepat, dan tidak terputus melalui tumpukan pemanggilan fungsi (*call stack*) rekursif alami.

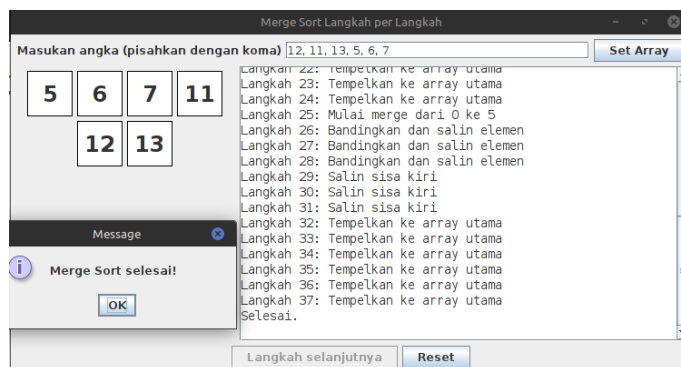
Namun, agar pergerakan data dapat dinikmati secara visual per langkah oleh pengguna melalui interaksi tombol, sifat rekursif alami tersebut harus dipecah. Kode ini berhasil mengatasi tantangan tersebut dengan cara melakukan pra-generasi langkah pembagian indeks menggunakan fungsi rekursif `generateMergeSteps_2022` di awal proses, lalu menyimpannya ke dalam struktur data antrean FIFO (*First-In, First-Out*). Antrean inilah yang kemudian dibaca secara sekuensial dan bertahap oleh mesin state di dalam method pemroses langkah, mengubah sifat algoritma yang tadinya beroperasi secara sinkronus-otomatis menjadi sistem simulasi interaktif berbasis state yang sangat efektif sebagai media pembelajaran visual.

Output :

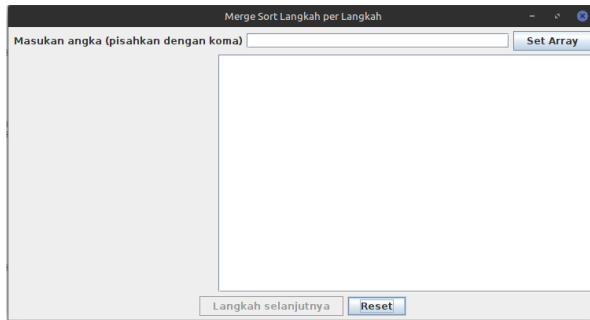
1. Angka dimasukkan dipisah dengan tanda koma lalu tekan tombol Set Array.



2. Tekan tombol Langkah selanjutnya untuk memulai pengurutan langkah per langkah hingga popup pemberitahuan proses pengurutan selesai



3. Tekan tombol Reset untuk mereset array.



2.5 Class BubbleSortGUI_2511532022.java

Class ini mengimplementasikan animasi visual interaktif dari algoritma *Bubble Sort*, menampilkan proses pembandingan dan pertukaran elemen yang bertetangga secara bertahap menggunakan skema pewarnaan dinamis pada komponen label antarmuka.

2.5.1 Kode program

```
1 package pekan8_2511532022;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.FlowLayout;
7 import java.awt.Font;
8
9 import javax.swing.BorderFactory;
10 import javax.swing.JButton;
11 import javax.swing.JFrame;
12 import javax.swing.JLabel;
13 import javax.swing.JOptionPane;
14 import javax.swing.JPanel;
15 import javax.swing.JScrollPane;
16 import javax.swing.JTextArea;
17 import javax.swing.JTextField;
18 import javax.swing.SwingConstants;
19 import javax.swing.SwingUtilities;
20
21 public class BubbleSortGUI_2511532022 extends JFrame {
22     private static final long serialVersionUID = 1L;
23     private int[] array_2022;
24     private JLabel[] labelArray_2022;
25     private JButton stepButton_2022, resetButton_2022, setButton_2022;
26     private JTextField inputField_2022;
27     private JPanel panelArray_2022;
28     private JTextArea stepArea_2022;
29
30     private int i_2022 = 0, j_2022 = 0;
31     private boolean sorting_2022 = false;
32     private int stepCount_2022 = 1;
33
34     public BubbleSortGUI_2511532022() {
35         setTitle("Bubble Sort Langkah per Langkah");
36         setSize(750, 400);
37         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
38         setLocationRelativeTo(null);
39         setLayout(new BorderLayout());
40
41         // panel input
42         JPanel inputPanel_2022 = new JPanel(new FlowLayout());
```



```

126     }
127
128     resetHighlights_2022();
129     StringBuilder stepLog_2022 = new StringBuilder();
130
131     labelArray_2022[j_2022].setBackground(Color.CYAN);
132     labelArray_2022[j_2022 + 1].setBackground(Color.CYAN);
133
134     if (array_2022[j_2022] > array_2022[j_2022 + 1]) {
135         // Swap
136         int temp_2022 = array_2022[j_2022];
137         array_2022[j_2022] = array_2022[j_2022 + 1];
138         array_2022[j_2022 + 1] = temp_2022;
139         labelArray_2022[j_2022].setBackground(Color.RED);
140         labelArray_2022[j_2022 + 1].setBackground(Color.RED);
141         stepLog_2022.append("Langkah ").append(stepCount_2022).append(": Menukar elemen ke-")
142             .append(j_2022).append(" ").append(array_2022[j_2022 + 1]).append(" dengan ke-")
143             .append(j_2022 + 1).append(" ").append(array_2022[j_2022]).append("\n");
144     } else {
145         stepLog_2022.append("Langkah ").append(stepCount_2022).append(": Tidak ada pertukaran antara ke-")
146             .append(j_2022).append(" dan ke-").append(j_2022 + 1).append("\n");
147     }
148
149     stepLog_2022.append("Hasil: ").append(arrayToString_2022(array_2022)).append("\n\n");
150     stepArea_2022.append(stepLog_2022.toString());
151     updateLabels_2022();
152
153     j_2022++;
154     if (j_2022 >= array_2022.length - i_2022 - 1) {
155         j_2022 = 0;
156         i_2022++;
157     }
158     stepCount_2022++;
159
160     if (i_2022 >= array_2022.length - 1) {
161         sorting_2022 = false;
162         stepButton_2022.setEnabled(false);
163         JOptionPane.showMessageDialog(this, "Sorting selesai!");
164     }
165 }
166
167 private void updateLabels_2022() {

```

```

168     for (int k = 0; k < array_2022.length; k++) {
169         labelArray_2022[k].setText(String.valueOf(array_2022[k]));
170     }
171 }
172
173 private void resetHighlights_2022() {
174     for (JLabel label : labelArray_2022) {
175         label.setBackground(Color.WHITE);
176     }
177 }
178
179 private void reset_2022() {
180     inputField_2022.setText("");
181     panelArray_2022.removeAll();
182     panelArray_2022.revalidate();
183     panelArray_2022.repaint();
184     stepArea_2022.setText("");
185     stepButton_2022.setEnabled(false);
186     sorting_2022 = false;
187     i_2022 = 0;
188     j_2022 = 0;
189     stepCount_2022 = 1;
190 }
191
192 private String arrayToString_2022(int[] arr) {
193     StringBuilder sb_2022 = new StringBuilder();
194     for (int k = 0; k < arr.length; k++) {
195         sb_2022.append(arr[k]);
196         if (k < arr.length - 1) sb_2022.append(", ");
197     }
198     return sb_2022.toString();
199 }
200
201 public static void main(String[] args) {
202     SwingUtilities.invokeLater(() -> {
203         BubbleSortGUI_2511532022 gui = new BubbleSortGUI_2511532022();
204         gui.setVisible(true);
205     });
206 }
207 }

```

2.5.2 Langkah kerja dan Penjelasan Kode

Berikut adalah penjelasan fungsi setiap baris kodenya

- Aplikasi desktop dijalankan lewat method main menggunakan utilitas penanganan thread grafis Java Swing via perintah `SwingUtilities.invokeLater` untuk menampilkan frame jendela utama di tengah layar monitor pengguna.
- Konstruktor objek mempersiapkan komponen-komponen antarmuka visual terstruktur, mendaftarkan fungsi pemantau aksi tombol (*action listener*), mengonfigurasi dimensi lebar tinggi jendela, serta menyusun peletakan tata ruang panel input teks, panel display grafis array, area teks riwayat langkah, dan tombol navigasi kontrol.
- Pengguna menginput sekumpulan deretan bilangan bulat acak dengan pemisah karakter koma pada kolom teks, lalu memicu method `setArrayFromInput_2022` dengan menekan tombol perintah "Set Array".
- Method penyiapan data mengurai teks inputan menggunakan fungsi pemotong string `split(",")`, melakukan konversi tipe data teks ke integer dalam blok proteksi penanganan error, serta menginisialisasi ulang seluruh variabel kontrol penanda perulangan meliputi `i_2022 = 0`, `j_2022 = 0`, dan variabel hitung langkah `stepCount_2022 = 1`.
- Panel visualisasi tengah dikosongkan terlebih dahulu, kemudian diisi ulang secara dinamis dengan deretan objek komponen grafis `JLabel` yang memuat nilai angka data input, diberi hiasan garis pembatas hitam, serta diset warna dasarnya menjadi putih bersih. Tombol navigasi langkah diaktifkan untuk memulai jalannya simulasi.
- Ketika pengguna melakukan aksi klik pada tombol "Langkah selanjutnya", method pengendali simulasi utama bernama `performStep_2022` dipicu untuk mengeksekusi satu tahapan perbandingan data. Langkah awal dimulai dengan mereset warna seluruh label menjadi putih melalui pemanggilan rutin method `resetHighlights_2022`.

- Program memeriksa status validitas perulangan luar; jika indeks penanda `i_2022` sudah mencapai batas panjang array dikurangi satu, maka proses dihentikan. Jika masih valid, dua buah label grafis `JLabel` yang posisinya saling bertetangga yaitu pada indeks berjalan `j_2022` dan indeks `j_2022 + 1` diubah warna latar belakang komponennya menjadi warna biru muda (Cyan) untuk menandakan kedua angka tersebut sedang dianalisis oleh sistem.
- Struktur kondisi logis memeriksa nilai data internal; jika angka pada indeks sebelah kiri (`array_2022[j_2022]`) bernilai lebih besar daripada nilai angka di indeks sebelah kanannya (`array_2022[j_2022 + 1]`), maka operasi pertukaran data (*swap*) mutlak dijalankan di dalam memori.
- Pada saat terjadi operasi penukaran tempat data tersebut, warna latar belakang kedua komponen `JLabel` diubah seketika menjadi warna merah terang untuk menegaskan terjadinya pergeseran nilai secara visual. Log deskripsi teks terperinci mengenai operasi pertukaran beserta posisi indeksnya dituliskan dan ditambahkan ke komponen `stepArea_2022`.
- Jika kondisi pemeriksaan nilai kiri tidak lebih besar dari nilai kanan, operasi penukaran data dilewati, komponen label tetap berwarna biru muda, dan sistem mencetak log riwayat informasi teks yang menyatakan bahwa tidak ada proses pertukaran data yang terjadi pada tahapan langkah aktif tersebut.
- Fungsi pembaruan tampilan bernama `method updateLabels_2022` dipanggil untuk menyinkronkan tulisan teks angka pada label grafis sesuai dengan kondisi nilai array memori terbaru, lalu nilai indeks penanda internal `j_2022` dinaikkan satu poin (`j_2022++`) untuk menggeser fokus perbandingan ke sepasang elemen berikutnya.
- Sistem memeriksa posisi pointer; jika nilai indeks `j_2022` telah mencapai batas akhir pengerjaan fase iterasi aktif (`array_2022.length - i_2022 - 1`), maka nilai koordinat indeks `j_2022` dikembalikan menjadi nol dan nilai indeks perulangan luar `i_2022` dinaikkan satu poin. Proses simulasi terus berlanjut di

setiap klik tombol hingga kondisi seluruh data terurut rapi, memicu munculnya dialog pop-up konfirmasi "Sorting selesai!", dan menonaktifkan tombol langkah. Pengguna dapat mengosongkan antarmuka kapan saja dengan menekan tombol "Reset".

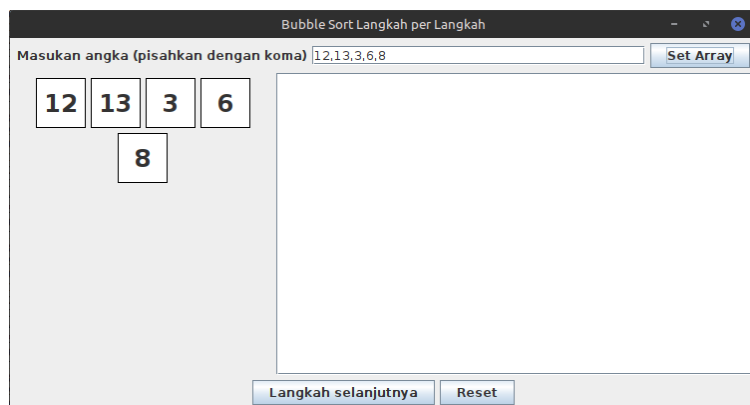
2.5.3 Analisis

Algoritma *Bubble Sort* merupakan metode pengurutan paling mendasar dengan mekanisme gelembung, di mana nilai berukuran besar akan perlahan bergeser naik ke posisi ujung kanan array di setiap akhir siklus internal perulangan. Melalui implementasi berbasis GUI pada kelas ini, kelemahan mendasar dari *Bubble Sort* dalam hal efisiensi waktu eksekusi yang bernilai $O(n^2)$ menjadi sangat terlihat jelas.

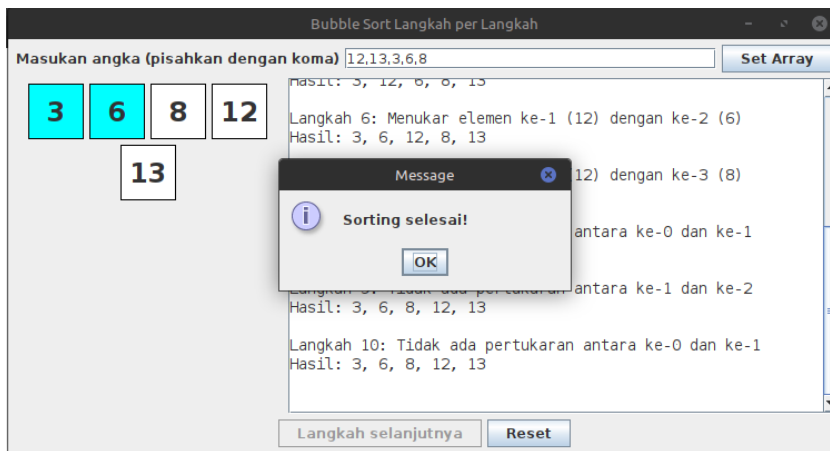
Pengguna dapat mengamati bahwa sistem harus melakukan pemindaian elemen bertetangga berulang-ulang dari awal indeks untuk setiap satu nilai gelembung tunggal yang berhasil ditempatkan di posisi kanan. Walaupun performa komputasinya lambat untuk data skala besar, visualisasi interaktif ini memberikan pemahaman struktural yang luar biasa jelas mengenai konsep penataan memori linear, manajemen indeks array, serta penanganan siklus perulangan bertingkat di dalam pengembangan sistem pemrograman.

Output :

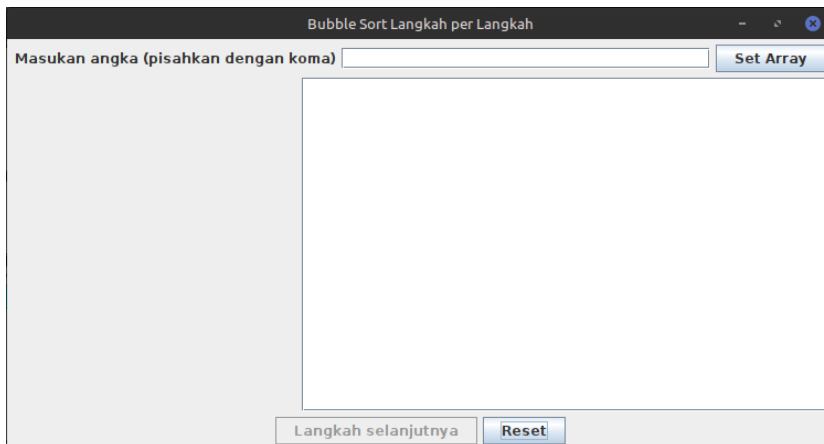
1. Angka dimasukkan dipisah dengan tanda koma lalu tekan tombol Set Array.



2. Tekan tombol Langkah selanjutnya untuk memulai pengurutan langkah per langkah hingga popup pemberitahuan proses pengurutan selesai



3. Tekan tombol Reset untuk mereset array.



BAB III

PENUTUP

Algoritma pengurutan memiliki karakteristik performa unik yang sangat bergantung pada struktur rancangan mekanismenya; *Shell Sort* unggul dalam kesederhanaan modifikasi jarak perbandingan, *Quick Sort* menawarkan kecepatan eksekusi tinggi berbasis elemen pembagi pivot, sementara *Merge Sort* menjamin konsistensi performa konstan melalui pembelahan elemen yang teratur.

- Pemilihan penggunaan algoritma pengurutan harus mempertimbangkan aspek ketersediaan sumber daya perangkat keras komputer; jika sistem mengutamakan penghematan penggunaan ruang memori RAM maka algoritma *Quick Sort* atau *Shell Sort* merupakan opsi ideal, sedangkan jika kestabilan waktu respons eksekusi menjadi prioritas utama tanpa memedulikan konsumsi memori tambahan maka algoritma *Merge Sort* merupakan pilihan terbaik.
- Penerapan konsep pemrograman antarmuka grafis (GUI) menggunakan pustaka Java Swing terbukti secara efektif mampu mentransformasikan instruksi logika kode program pengurutan yang bersifat abstrak dan teoritis menjadi bentuk visualisasi pergerakan data interaktif yang sangat mudah dipahami alur kerjanya.
- Penggunaan teknik antrean linier (*Queue*) dan manipulasi variabel penanda state internal merupakan solusi rekayasa perangkat lunak yang sangat andal untuk mengontrol pemisahan tahapan jalannya eksekusi fungsi rekursif otomatis agar dapat dikendalikan secara interaktif per langkah melalui penekanan tombol antarmuka pengguna.

Tugas Praktikum Struktur Data

Pekan 8

Topik: Algoritma Sorting (Shell, Quick, dan Merge)

Tema: Mengurutkan Playlist Lagu

Berikut ini telah mencakup ketiga algoritma (**Shell Sort**, **Quick Sort**, dan **Merge Sort**) yang bekerja secara penuh, menggunakan perbaikan objek input System.in, serta menerapkan aturan penamaan berbasis NIM (**2511532022** dan ujung **2022**) secara konsisten pada setiap komponen variabel dan method.

1. Kode Program

1.1 Class Lagu_2511532022.java (ADT)

```
1 package pekan8_2511532022;
2
3 public class Lagu_2511532022 {
4
5     private String judul_2022;
6     private String penyanyi_2022;
7     private int durasi_2022; // Dalam satuan detik
8
9     public Lagu_2511532022(String judul_2022, String penyanyi_2022, int durasi_2022) {
10         this.judul_2022 = judul_2022;
11         this.penyanyi_2022 = penyanyi_2022;
12         this.durasi_2022 = durasi_2022;
13     }
14
15     public String getJudul_2022() {
16         return judul_2022;
17     }
18
19     public String getPenyanyi_2022() {
20         return penyanyi_2022;
21     }
22
23     public int getDurasi_2022() {
24         return durasi_2022;
25     }
26 }
```


1.2 Class Sorting_2511532022.java

```
1 package pekan8_2511532022;
2
3 import java.util.Scanner;
4
5 public class Sorting_2511532022 {
6
7     // Array untuk menyimpan maksimal 20 lagu sesuai ketentuan tugas
8     private Lagu_2511532022[] dataLagu_2022 = new Lagu_2511532022[20];
9     private int jumlahLagu_2022 = 0;
10
11     // Method untuk mengisi data awal (minimal 7 lagu)
12     public void inputData_2022() {
13         dataLagu_2022[0] = new Lagu_2511532022("Mio Cristo Piange Diamanti", "Musisi A", 270);
14         dataLagu_2022[1] = new Lagu_2511532022("La Rumba Del Perdon", "Musisi B", 252);
15         dataLagu_2022[2] = new Lagu_2511532022("La Perla", "Musisi C", 196);
16         dataLagu_2022[3] = new Lagu_2511532022("Hati-Hati di Jalan", "Tulus", 242);
17         dataLagu_2022[4] = new Lagu_2511532022("Gajah", "Tulus", 221);
18         dataLagu_2022[5] = new Lagu_2511532022("Monokrom", "Tulus", 214);
19         dataLagu_2022[6] = new Lagu_2511532022("Sial", "Mahalini", 243);
20         jumlahLagu_2022 = 7;
21     }
22
23     // Fungsi pembantu tukar posisi elemen objek Lagu
24     private void swap_2022(int i, int j) {
25         Lagu_2511532022 temp_2022 = dataLagu_2022[i];
26         dataLagu_2022[i] = dataLagu_2022[j];
27         dataLagu_2022[j] = temp_2022;
28     }
29
30     // ===== 1. IMPLEMENTASI SHELL SORT =====
31     // Shell Sort mengurutkan berdasarkan JUDUL secara alfabetis (A-Z)
32     public void shellSort_2022() {
33         int n_2022 = jumlahLagu_2022;
34         int gap_2022 = n_2022 / 2;
35
36         while (gap_2022 > 0) {
37             for (int i_2022 = gap_2022; i_2022 < n_2022; i_2022++) {
38                 Lagu_2511532022 temp_2022 = dataLagu_2022[i_2022];
39                 int j_2022 = i_2022;
40
41                 // Bandingkan string judul menggunakan compareTo untuk urutan A-Z
42                 while (j_2022 >= gap_2022 &&
43                     dataLagu_2022[j_2022 - gap_2022].getJudul_2022().compareTo(temp_2022.getJudul_2022()) > 0) {
44                     dataLagu_2022[j_2022] = dataLagu_2022[j_2022 - gap_2022];
45                     j_2022 = j_2022 - gap_2022;
46                 }
47                 dataLagu_2022[j_2022] = temp_2022;
48             }
49             gap_2022 = gap_2022 / 2;
50         }
51     }
52
53     // ===== 2. IMPLEMENTASI QUICK SORT =====
54     // Logika partisi Quick Sort berdasarkan Durasi (Ascending)
55     private int partition_2022(int low_2022, int high_2022) {
56         int pivot_2022 = dataLagu_2022[high_2022].getDurasi_2022();
57         int i_2022 = (low_2022 - 1);
58
59         for (int j_2022 = low_2022; j_2022 < high_2022; j_2022++) {
60             if (dataLagu_2022[j_2022].getDurasi_2022() < pivot_2022) {
61                 i_2022++;
62                 swap_2022(i_2022, j_2022);
63             }
64         }
65         swap_2022(i_2022 + 1, high_2022);
66         return (i_2022 + 1);
67     }
68
69     // Method Utama Quick Sort Rekursif
70     public void quickSort_2022(int low_2022, int high_2022) {
71         if (low_2022 < high_2022) {
72             int pi_2022 = partition_2022(low_2022, high_2022);
73             quickSort_2022(low_2022, pi_2022 - 1);
74             quickSort_2022(pi_2022 + 1, high_2022);
75         }
76     }
77
78     // ===== 3. IMPLEMENTASI MERGE SORT =====
79     // Fungsi penggabungan untuk Merge Sort (Berdasarkan JUDUL A-Z)
80     private void merge_2022(int l_2022, int m_2022, int r_2022) {
81         int n1_2022 = m_2022 - l_2022 + 1;
```

```

82     int n2_2022 = r_2022 - m_2022;
83
84     Lagu_2511532022[] L_2022 = new Lagu_2511532022[n1_2022];
85     Lagu_2511532022[] R_2022 = new Lagu_2511532022[n2_2022];
86
87     for (int i = 0; i < n1_2022; ++i) {
88         L_2022[i] = dataLagu_2022[l_2022 + i];
89     }
90     for (int j = 0; j < n2_2022; ++j) {
91         R_2022[j] = dataLagu_2022[m_2022 + 1 + j];
92     }
93
94     int i_2022 = 0, j_2022 = 0;
95     int k_2022 = l_2022;
96
97     while (i_2022 < n1_2022 && j_2022 < n2_2022) {
98         // Membandingkan judul alfabetis (A-Z)
99         if (L_2022[i_2022].getJudul_2022().compareTo(R_2022[j_2022].getJudul_2022()) <= 0) {
100             dataLagu_2022[k_2022] = L_2022[i_2022];
101             i_2022++;
102         } else {
103             dataLagu_2022[k_2022] = R_2022[j_2022];
104             j_2022++;
105         }
106         k_2022++;
107     }
108
109     while (i_2022 < n1_2022) {
110         dataLagu_2022[k_2022] = L_2022[i_2022];
111         i_2022++;
112         k_2022++;
113     }
114
115     while (j_2022 < n2_2022) {
116         dataLagu_2022[k_2022] = R_2022[j_2022];
117         j_2022++;
118         k_2022++;
119     }
120 }
121
122 // Method Utama Merge Sort Rekursif

```

```

123 public void mergeSort_2022(int l_2022, int r_2022) {
124     if (l_2022 < r_2022) {
125         int m_2022 = (l_2022 + r_2022) / 2;
126
127         mergeSort_2022(l_2022, m_2022);
128         mergeSort_2022(m_2022 + 1, r_2022);
129
130         merge_2022(l_2022, m_2022, r_2022);
131     }
132 }
133
134 // Method untuk menampilkan isi data playlist lagu
135 public void tampilData_2022() {
136     for (int k_2022 = 0; k_2022 < jumlahLagu_2022; k_2022++) {
137         System.out.println((k_2022 + 1) + "." + dataLagu_2022[k_2022].getJudul_2022() +
138             " (" + dataLagu_2022[k_2022].getPenyanyi_2022() + ") - " +
139             dataLagu_2022[k_2022].getDurasi_2022() + " detik");
140     }
141 }
142
143 public static void main(String[] args_2022) {
144     Scanner scanner_2022 = new Scanner(System.in);
145     Sorting_2511532022 manager_2022 = new Sorting_2511532022();
146
147     // Memuat data awal lagu
148     manager_2022.inputData_2022();
149
150     System.out.println("Sorting Playlist NIM: 2511532022");
151     System.out.print("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): ");
152     int pilihan_2022 = scanner_2022.nextInt();
153
154     System.out.println("\nData Sebelum Sorting:");
155     manager_2022.tampilData_2022();
156
157     switch (pilihan_2022) {
158         case 1:
159             manager_2022.shellSort_2022();
160             System.out.println("\nData Setelah Shell Sort (Judul A-Z):");
161             manager_2022.tampilData_2022();
162             break;
163         case 2:

```

```

163-      case 2:
164          manager_2022.quickSort_2022(0, manager_2022.jumlahLagu_2022 - 1);
165          System.out.println("\nData Setelah Quick Sort (Durasi Asc):");
166          manager_2022.tampilData_2022();
167          break;
168-      case 3:
169          manager_2022.mergeSort_2022(0, manager_2022.jumlahLagu_2022 - 1);
170          System.out.println("\nData Setelah Merge Sort (Judul A-Z):");
171          manager_2022.tampilData_2022();
172          break;
173-      default:
174          System.out.println("\nPilihan tidak valid!");
175          break;
176      }
177
178      scanner_2022.close();
179  }
180 }

```

2. Langkah Kerja

Langkah kerja atau Alur program telah sesuai dengan apa yang diberitahu pada file penugasan dan informasi ralat detail tugas pada pesan grup pratikum struktur data.

Pada Class Lagu_2511532022 program mendefinisikan kelas blueprint dalam ruang nama package pekan8_2511532022.

- Mendeklarasikan tiga buah atribut bertipe privat guna menerapkan prinsip enkapsulasi data: judul_2022 (String), penyanyi_2022 (String), dan durasi_2022 (int).
- Membuat fungsi kontruktor Lagu_2511532022 berparameter lengkap untuk menerima data masukan dari luar dan menetapkannya langsung ke atribut internal instansiasi objek menggunakan kata kunci referensi this.
- Menyediakan metode pengakses publik (*getter method*) yang terdiri atas getJudul_2022(), getPenyanyi_2022(), dan getDurasi_2022() untuk mengembalikan nilai parameter objek ketika dibutuhkan oleh kelas sorting utama.

Langkah kerja Eksekusi :

1. Program memulai eksekusi dari dalam fungsi utama main(), di mana objek Scanner dikonfigurasi menggunakan stream System.in untuk membaca input konsol.
2. Instansiasi objek pengelola manager_2022 dibentuk di memori heap, kemudian langsung memanggil method inputData_2022() untuk memuat 7 objek lagu awal ke dalam array dataLagu_2022 serta mengubah penanda ukuran indeks jumlahLagu_2022 menjadi 7.
3. Judul identitas tugas dicetak ke layar konsol, dan pengguna diminta memasukkan angka integer penentu pilihan operasi algoritma sorting melalui scanner_2022.nextInt().
4. Keadaan array playlist yang belum teratur dicetak secara runut menggunakan rutin iterasi method tampilData_2022().
5. Struktur kendali switch(pilihan_2022) memotong alur eksekusi berdasarkan input pengguna:
 - **Jika memilih Opsi 1 (Shell Sort):** Program melompat ke method shellSort_2022(). Sistem membagi total data dengan interval $gap_2022 = n_2022 / 2$. Selama gap lebih besar dari 0, program melakukan pengurutan penyisipan sub-array dengan membandingkan string alfabetis judul lagu memakai fungsi bawaan .compareTo(). Objek ditukar jika judul sebelah kiri memiliki urutan karakter lebih tinggi (A-Z). Interval gap kemudian diperkecil berulang kali ($gap_2022 /= 2$) sampai seluruh playlist terurut rapi.
 - **Jika memilih Opsi 2 (Quick Sort):** Program memicu parameter utama quickSort_2022(0, 6). Fungsi rekursif mengecek validasi batas penjelajahan $low_2022 < high_2022$ lalu memanggil method jembatan partisi partition_2022. Di dalam partisi, durasi lagu objek terujung kanan dijadikan acuan batas (pivot_2022). Iterasi internal

membandingkan durasi setiap lagu terhadap pivot, lalu menukar elemen-elemen kecil ke sisi kiri pivot menggunakan bantuan method pointer `swap_2022()`. Melalui strategi *divide and conquer*, sub-array kiri dan kanan diproses ulang secara rekursif hingga data terurut menaik (*ascending*) berdasarkan waktu detik.

- **Jika memilih Opsi 3 (Merge Sort):** Program memicu pemisahan rekursif biner lewat method `mergeSort_2022(0, 6)`. Array dipecah menjadi dua bagian di titik tengah (`m_2022`), diproses secara rekursif hingga berukuran elemen tunggal, lalu digabungkan kembali dalam keadaan terurut alfabetis (Judul A-Z) melalui fungsi `merge_2022()`. Di dalam fungsi `merge`, diciptakan array penampung sementara (`L_2022` dan `R_2022`) untuk membandingkan judul dari kedua belahan sebelum disalin kembali ke dalam array utama.

6. Hasil susunan playlist akhir pasca-sorting langsung dicetak ulang ke layar melalui fungsi `tampilData_2022()` dan objek pembaca input ditutup aman lewat `scanner_2022.close()`.

3. Hasil Pengujian (*Output*)

Saya memilih Quick Sort karena algoritma ini memiliki kompleksitas waktu rata-rata (*average case*) sebesar $O(n \log n)$, yang menjadikannya salah satu algoritma pengurutan internal tercepat dalam memori RAM komputer. Karena melakukan pertukaran posisi elemen langsung pada array referensi asli (*in-place*),

Berdasarkan informasi update tugas pada Whatsapp semua algoritma dibuat, dan saya memilih satu algoritma sorting untuk laporan. Berikut Screenshot Pilihan Pengurutan (*sorting quick sort*) :

Sorting Playlist NIM: 2511532022
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2

Data Sebelum Sorting:

1. Mio Cristo Piange Diamanti (Musisi A) - 270 detik
2. La Rumba Del Perdon (Musisi B) - 252 detik
3. La Perla (Musisi C) - 196 detik
4. Hati-Hati di Jalan (Tulus) - 242 detik
5. Gajah (Tulus) - 221 detik
6. Monokrom (Tulus) - 214 detik
7. Sial (Mahalini) - 243 detik

Data Setelah Quick Sort (Durasi Asc):

1. La Perla (Musisi C) - 196 detik
2. Monokrom (Tulus) - 214 detik
3. Gajah (Tulus) - 221 detik
4. Hati-Hati di Jalan (Tulus) - 242 detik
5. Sial (Mahalini) - 243 detik
6. La Rumba Del Perdon (Musisi B) - 252 detik
7. Mio Cristo Piange Diamanti (Musisi A) - 270 detik